`scripts/run-helixqa-challenges.sh` — User Guide

Last verified: 2026-05-16 (Phase 4 follow-up B — 4 open questions resolved)
Inheritance: HelixConstitution §11.4.18 (script documentation mandate) + Lava §6.AE (Comprehensive Challenge Coverage) + §6.J (Anti-Bluff Functional Reality) + §6.X (Container-Submodule Emulator Wiring — implemented via `--runner=containerized`) + §6.W (mirror-host boundary — per-script audit at `docs/helixqa-script-audit.md`) **Classification:** project-specific (the per-script wiring choices + the §6.W audit results are Lava-specific; the host-vs-container delegation is universal per §6.X)

Overview

Thin Lava-side wrapper around HelixQA's 11 Challenge scripts shipped at `submodules/helixqa/challenges/scripts/`. Invokes them in sequence, captures per-script stdout/stderr to log files, writes a roll-up attestation JSON, and reports per-script PASS / FAIL / SKIP outcomes.

Implements **Option 1** from `docs/plans/2026-05-16-helixqa-integration-design.md` — shell-level wiring with NO modification to HelixQA. Future cycles MAY:

- Add Go-package linking for Group A packages (Option 2 in the design).
- Migrate Compose UI Challenge Tests to use HelixQA as backend (Option 3 in the design — deferred indefinitely).

The 11 wired scripts

Canonical list (matches `HELIXQA_SCRIPTS` array in the script body):

#	Script	What it checks (HelixQA's own framing)	Toolchain
1	`anchor_manifest_challenge.sh`	`docs/behavior-anchors.md` rows + per-anchor file/symbol resolution	none
2	`bluff_scanner_challenge.sh`		**go**

#	Script	What it checks (HelixQA's own framing)	Toolchain
		scanner self-test + tree-wide bluff scan	
3	`chaos_failure_injection_challenge.sh`	chaos-engineering harness	none
4	`ddos_health_flood_challenge.sh`	DDoS-resilience health-endpoint flooding	none
5	`host_no_auto_suspend_challenge.sh`	host suspend / hibernate / sleep targets masked + sleep.conf override (Linux-only)	none
6	`mutation_ratchet_challenge.sh`	go-mutesting kill-rate ratchet	**go**
7	`no_suspend_calls_challenge.sh`	source-tree scan for forbidden host-power-management invocations	none
8	`scaling_horizontal_challenge.sh`	horizontal-scaling load test	none
9	`stress_sustained_load_challenge.sh`	sustained-load stress test	none
10	`ui_terminal_interaction_challenge.sh`	terminal UI interaction (limited Lava applicability)	none
11	`ux_end_to_end_flow_challenge.sh`	end-to-end UX flow harness	none

Toolchain column source: `HELIXQA_TOOLCHAIN_MAP` constant in the wrapper. `go` means the script invokes a Go binary (go-mutesting, the bluff scanner) that requires `go` on PATH.

Usage

```
# Run ALL 11 HelixQA Challenge scripts on the HOST (workstation
        iteration)
bash scripts/run-helixqa-challenges.sh
```

```
# Run only specific scripts (comma-separated basenames
        without .sh)
bash scripts/run-helixqa-challenges.sh --only
        host_no_auto_suspend_challenge,no_suspend_calls_challenge

# Custom evidence dir
bash scripts/run-helixqa-challenges.sh --evidence-dir .lava-ci-
        evidence/myrun/helixqa

# Custom evidence dir via env var (useful when wrapping)
LAVA_HELIXQA_EVIDENCE_DIR=/tmp/helixqa bash scripts/run-helixqa-
        challenges.sh

# Halt the loop on first non-zero exit (default: continue)
bash scripts/run-helixqa-challenges.sh --stop-on-fail

# Suppress stdout summary (machine consumption only)
bash scripts/run-helixqa-challenges.sh --json-only

# §6.X containerized runner (gate-mode evidence)
bash scripts/run-helixqa-challenges.sh --runner containerized --
        container-image docker.io/library/golang:1.22

# Skip Go-requiring scripts when Go is unavailable
bash scripts/run-helixqa-challenges.sh --require-toolchain none
```

Inputs

Arg / Env	Description
`--evidence-dir <dir>`	Output directory (default: `$LAVA_HELIXQA_EVIDENCE_DIR` if set; else `.lava-ci-evidence/helixqa-challenges/<UTC-timestamp>/`)
`LAVA_HELIXQA_EVIDENCE_DIR` env	Env-var alternative to `--evidence-dir`. CLI flag wins if both are set.
`--only NAME1,NAME2,...`	Run only the listed scripts (basename without `.sh`); errors if none match
`--continue-on-fail`	Keep running scripts after a FAIL (default behavior)
`--stop-on-fail`	Halt the loop on first FAIL (useful for triage of cascading failures)
`--json-only`	Suppress the human-readable stdout summary; only write the attestation JSON
`--runner host|containerized`	§6.X delegation: `host` (default — workstation iteration) or `containerized` (required for §6.AE gate runs). `containerized` honest-fail-fasts (exit 4) when `submodules/containers` is absent.
`--container-image <image>`	

Arg / Env	Description
	Container image for `--runner=containerized`. Default: `docker.io/library/golang:1.22` (provides Go toolchain for HelixQA's Go-requiring scripts).
`--container-runtime podman|docker`	Container runtime for `--runner=containerized`. Default: auto-detect (podman preferred per §6.U).
`--require-toolchain go|none`	§6.J toolchain precondition. `go` (default): wrapper exits 4 if `go` is absent from PATH (host mode) or always available (container mode). `none`: Go-requiring scripts (bluff_scanner, mutation_ratchet) SKIP with a clear precondition message when `go` is absent.

Outputs

```
<evidence-dir>/
├── helixqa-attestation.json          # roll-up: pin, runner,
totals, per-script rows
├── anchor_manifest_challenge.log     # per-script stdout+stderr
├── bluff_scanner_challenge.log
├── chaos_failure_injection_challenge.log
├── ddos_health_flood_challenge.log
├── host_no_auto_suspend_challenge.log
├── mutation_ratchet_challenge.log
├── no_suspend_calls_challenge.log
├── scaling_horizontal_challenge.log
├── stress_sustained_load_challenge.log
├── ui_terminal_interaction_challenge.log
└── ux_end_to_end_flow_challenge.log
```

Attestation JSON shape


```json
{
  "timestamp": "2026-05-16T03:52:15Z",
  "helixqa_pin": "403603dbd47b971549709bd9f2efa30dca810097",
  "helixqa_runner": "host",
  "helixqa_runner_caveat": "§6.X container-wrapping NOT in use
        this run; rerun with --runner=containerized for §6.AE
        gate-mode evidence",
  "require_toolchain": "go",
  "toolchain_available": true,
  "container_runtime": "n/a",
  "container_image": "n/a",
  "w_excluded_count": 0,
  "w_exclusion_audit": "docs/helixqa-script-audit.md",
  "total_scripts": 11,
  "pass_count": 5,
```

```
  "fail_count": 2,
  "skip_count": 4,
  "halted_early": false,
  "all_passed": false,
  "scripts": [
    {"name": "anchor_manifest_challenge.sh", "result": "FAIL",
        "exit_code": 1, "duration_seconds": 0, "log":
        "anchor_manifest_challenge.log"}
  ]
}
```

When `--runner=containerized` is selected the `helixqa_runner` field reads `"containerized"`, the caveat is replaced with `"(no caveat — §6.X containerized runner active)"`, and `container_runtime` / `container_image` carry the actual values used.

Exit codes

Code	Meaning
0	All invoked scripts returned 0 (or `SKIP` exit 2). Zero `FAIL` outcomes.
1	One or more invoked scripts returned non-zero exit other than the SKIP-canonical 2. Real defect surfaced by HelixQA OR `--stop-on-fail` halted the loop.
2	Invalid arguments (`--only` matched no scripts; unknown flag; invalid `--runner` or `--require-toolchain` value).
3	Missing dependency — `submodules/helixqa` directory absent, OR the canonical 11-script wired list drifted from what the pin actually ships.
4	Missing runtime — `--runner=containerized` without `submodules/containers` bootstrapped, OR `--runner=containerized` without a container runtime on PATH, OR `--require-toolchain=go` without `go` on PATH.

The 0/1/2/3/4 split is **anti-bluff load-bearing**: exit 0 means scripts genuinely ran with no failures; exit 3 means we honestly cannot claim to have run them (precondition gap); exit 4 means a runtime / toolchain dependency is missing (silent degradation forbidden per §6.J). Per §6.J we never silently mask a precondition gap as "success".

Per-script SKIP / FAIL semantics

The wrapper treats each script's own exit codes as authoritative and classifies:

- `exit 0` → **PASS** — script reports success
- `exit 2` → **SKIP** — script's documented "scanner missing / precondition unmet" exit code (per `no_suspend_calls_challenge.sh` convention)
- any other non-zero → **FAIL** — real defect OR Linux-only check failing on macOS/other host

Additionally the wrapper itself may classify a script as SKIP BEFORE invoking it:

- The script is on the `HELIXQA_W_EXCLUSIONS` array (§6.W audit found a violation) → **SKIP** exit-2 with a per-script log line citing the audit doc
- The script declares `toolchain: go` in `HELIXQA_TOOLCHAIN_MAP` AND `--require-toolchain=none` AND `go` absent from PATH → **SKIP** exit-2 with a precondition-unmet message

Many of the 11 HelixQA scripts are LINUX-SPECIFIC (they invoke `systemctl`, read `/etc/systemd/`, parse `journalctl`). On macOS or any host without systemd, these will FAIL — not because Lava is broken, but because the HelixQA author wrote them assuming a Linux host. **This is the honest behavior** — per §6.J the wrapper does NOT mis-classify "wrong-OS" as PASS or as SKIP without operator intervention. Operators on macOS hosts SHOULD use `--only` to filter to portable scripts (`anchor_manifest_challenge`, `bluff_scanner_challenge`, `mutation_ratchet_challenge` — when the HelixQA-internal scanner directory is present), OR use `--runner=containerized` to provide a Linux execution surface inside a podman/docker container.

§6.X containerized runner

When invoked with `--runner=containerized`, the wrapper:

1. Verifies `submodules/containers` is bootstrapped (else exit 4)
2. Auto-detects the container runtime (`podman` preferred; falls back to `docker`)
3. Selects the container image (default: `docker.io/library/golang:1.22` to provide the Go toolchain HelixQA's Go-requiring scripts need)
4. For each script, invokes
   `<runtime> run --rm --user $(id -u):$(id -g) -v $REPO_ROOT:$REPO_ROOT:rw -w $HELIXQA_SCRIPTS_DIR -e HOME=/tmp $CONTAINER_IMAGE bash ./<script>`
5. Writes the same per-script logs + attestation JSON to the evidence dir on the host filesystem (via the bind mount)

The container runs as the host UID so evidence files retain operator ownership — no root-owned outputs.

Limitations of the containerized runner (honestly disclosed):

- Scripts that probe HOST systemd state (`host_no_auto_suspend_challenge.sh`) will see the CONTAINER's systemd state, not the host's. Their results inside a container are NOT meaningful for the host the operator runs on; treat them as HelixQA's own self-test rather than as a Lava-host audit.
- Scripts that connect to LOCAL services (`chaos`, `ddos`, `scaling`, `stress` against `localhost:8081`) will hit the container's localhost, not the host's. The operator MUST either run the local service inside the same network namespace OR override `HELIXQA_HEALTH_URL` to a reachable address. The wrapper does NOT pre-validate this — it is operator-supplied per §6.W audit.

§6.W per-script mirror-policy audit

`docs/helixqa-script-audit.md` is the source-of-truth audit. As of the last audit (2026-05-16), 0 of 11 scripts violate §6.W on default config — the `HELIXQA_W_EXCLUSIONS` array is therefore empty. The audit MUST be re-run when the `submodules/helixqa` pin bumps; the wrapper's wiring-drift check (exit 3 on added/removed scripts) prompts re-audit but does NOT mechanize the per-script grep — that remains a human-driven audit per §6.J ("real grep results only — no manufactured findings").

Wiring into `scripts/run-challenge-matrix.sh`

The §6.AE Challenge matrix runner accepts an opt-in `--include-helixqa` flag that invokes this wrapper as a sibling step BEFORE the AVD matrix runs:

```
bash scripts/run-challenge-matrix.sh --include-helixqa
```

The flag is **OFF by default** so existing matrix invocations (operator iterations, prior CI flows) are unaffected. When enabled:

- HelixQA wrapper runs on the host FIRST (host independent of AVD matrix)
- Wrapper evidence lands at `<matrix-evidence-dir>/helixqa/`
- A non-zero HelixQA exit is folded into the matrix runner's final exit code (matrix exit dominates if both fail; HelixQA exit promotes matrix-0 to 1 if HelixQA reports FAIL)
- HelixQA does NOT run inside the §6.X-debt darwin/arm64 gate-host skip — it runs regardless because it doesn't need KVM

To force the matrix-aggregated HelixQA invocation to use the containerized runner, the operator currently runs the wrapper separately:

```
bash scripts/run-helixqa-challenges.sh --runner containerized
bash scripts/run-challenge-matrix.sh         # without --include-
        helixqa
```

A future cycle MAY teach the matrix runner to pass `--runner=containerized` through; today the matrix runner invokes the wrapper with default arguments (host mode) since the matrix runner itself only fully runs on Linux x86_64 hosts where the host runner is effectively equivalent.

Anti-bluff posture

Per `docs/plans/2026-05-16-helixqa-integration-design.md` §"6.J anti-bluff posture":

1. **Each invoked script MUST actually be invokable.** The wrapper's pre-flight rejects (exit 3) if the wired list drifts from the actual pin contents.
2. **Each result MUST be from a real execution.** Per-script log files are written from the actual subprocess output — captured stdout+stderr go to `<evidence-dir>/<script>.log`. The hermetic test `tests/check-constitution/`

`test_helixqa_wiring.sh` asserts the fixture's stdout string appears in the produced log.

3. **Per §6.AC non-fatal telemetry:** SKIP exits do NOT cause the wrapper itself to FAIL but ARE recorded distinctly from PASS. The attestation JSON's `skip_count` is operator-visible so a regression that flips many PASSes to SKIPs is detectable.
4. **Per §6.J/§6.L:** the wrapper exists; HelixQA-emitted FAILs are real defects to triage; HelixQA-emitted SKIPs are honest acknowledgments of precondition gaps — NOT false-pass coverage.
5. **Per §6.X:** `--runner=containerized` honest-fail-fasts (exit 4) when `submodules/containers` is absent. The wrapper NEVER silently degrades containerized → host because that would be the exact bluff §6.X exists to prevent.
6. **Per §6.J:** `--require-toolchain=go` (default) exits 4 when `go` is absent rather than silently SKIPping Go-requiring scripts. Silent-skip-with-default would produce false-green coverage; operators who knowingly work without Go MUST opt in via `--require-toolchain=none`.
7. **Per §6.W:** the per-script audit at `docs/helixqa-script-audit.md` is the binding source-of-truth for the `HELIXQA_W_EXCLUSIONS` array. Manufactured findings are forbidden; only real `grep` results may populate the audit.

Hermetic test

`tests/check-constitution/test_helixqa_wiring.sh` — 11 fixtures:

Fixture	Asserts
`test_passes_when_helixqa_present_and_all_green`	Wrapper exit 0; attestatio reports 11/0/0; per-script files exist + contain the fi stdout (proving subproces actually ran)
`test_fails_when_script_missing`	Wrapper exit 3 (wiring dr error message names the missing script
`test_skip_mode_when_helixqa_absent`	Wrapper exit 3 (missing submodule); error messag includes the `git submod update --init submod helixqa` remediation command; no attestation written
`test_fail_classified_correctly`	Wrapper exit 1 when one returns 1; attestation mark script as FAIL (not SKIP anti-bluff classification)
`test_containerized_runner_fails_fast_when_containers_absent`	Q1: Wrapper exit 4; error message names §6.X + g `submodule update --i submodules/containe` remediation

Fixture	Asserts
`test_host_runner_default_attestation_shape`	Q1: default-runner attesta has `helixqa_runner: h` caveat field + `require_toolchain: g` `w_exclusion_audit` poi
`test_require_toolchain_go_fails_fast_when_go_absent`	Q2: Wrapper exit 4 when absent + default `--requi` `toolchain=go`; error nar the `--require-` `toolchain=none` escape
`test_require_toolchain_none_skips_go_scripts_when_go_absent`	Q2: Wrapper exit 0; Go-requiring scripts (bluff_scanner, mutation_ratchet) marked SKIP; non-Go scripts stil PASS
`test_evidence_dir_env_var_override`	Q3: `LAVA_HELIXQA_EVIDENC` env var redirects attestati default dated dir is NOT populated
`test_invalid_runner_rejected`	Q1: Wrapper exit 2 + exp error on unknown `--runn` value
`test_invalid_require_toolchain_rejected`	Q2: Wrapper exit 2 + exp error on unknown `--requ` `toolchain` value

Run: `bash tests/check-constitution/test_helixqa_wiring.sh`

Open questions (resolved 2026-05-16 — Phase 4 follow-up B)

The four open questions from the initial Option 1 commit were resolved in this commit:

1. **Q1 — Container vs host runner** (§6.X integration): RESOLVED. New `--runner=host|containerized` flag (default: `host` for workstation iteration; `containerized` required for §6.AE gate runs). Containerized mode honest-fail-fasts (exit 4) when `submodules/containers` is absent — no silent degradation per §6.J.
2. **Q2 — Real-deps vs stub gating**: RESOLVED. New `--require-toolchain=go|none` flag (default: `go`). Per-script toolchain requirement declared in `HELIXQA_TOOLCHAIN_MAP` constant (bluff_scanner + mutation_ratchet need Go; the other 9 do not). `--require-toolchain=go` exits 4 when `go` absent; `--require-toolchain=none` SKIPs Go-requiring scripts with a clear precondition message.
3. **Q3 — Evidence-directory boundary**: RESOLVED. `--evidence-dir` flag is preferred; `LAVA_HELIXQA_EVIDENCE_DIR` env-var override added for parent-

runner wrapping use cases. Default location remains `.lava-ci-evidence/helixqa-challenges/<UTC-timestamp>/` per Lava convention.
4. **Q4 — §6.W mirror policy**: RESOLVED. Per-script audit landed at `docs/helixqa-script-audit.md`; `HELIXQA_W_EXCLUSIONS` array constant in the wrapper consumes the audit's findings. 0/11 scripts currently violate §6.W on default config (the array is empty); re-audit owed on every `submodules/helixqa` pin bump.

See this commit's body for the per-question implementation details + falsifiability rehearsals.

Cross-references

- `docs/plans/2026-05-16-helixqa-integration-design.md` — design doc (Option 1 chosen this cycle; Option 2 + Option 3 deferred)
- `docs/scripts/run-challenge-matrix.sh.md` — the `--include-helixqa` flag
- `docs/helix-constitution-gates.md` — `CM-HELIXQA-WIRING` + `CM-HELIXQA-§6.W-AUDIT` gate rows
- `docs/helixqa-script-audit.md` — §6.W per-script audit (Phase 4 follow-up B Q4 resolution)
- `submodules/helixqa/CLAUDE.md` — HelixQA's own anti-bluff covenant
- `submodules/helixqa/README.md` — HelixQA framework overview
- Lava `CLAUDE.md` §6.AE + §6.J + §6.X + §6.W + §6.AD (HelixConstitution inheritance)